

MiniLang Compiler Usage Guide

1 How to Run the Project

1.1 Compilation Steps

Open terminal in the project directory and run:

```
flex lexer.l
bison -d parser.y
gcc lex.yy.c parser.tab.c *.c -o minicompiler
```

1.2 Execution

Run the compiler:

```
./minicompiler < testcases/test1.ml
```

Output Files Generated:

- output.tac (Three Address Code)
- output.asm (Pseudo Assembly)

—

2 Input Format

The compiler accepts MiniLang programs with:

- Variable declarations (`int`, `bool`)
- Assignment statements
- Arithmetic expressions
- Conditional statements (`if-else`)
- Loop statements (`while`)
- Print statements

—

3 Output Format

- TAC format:

```
t0 = a + b
x = t0
```

- Assembly format:

```
LOAD r1, a
LOAD r2, b
ADD r3, r1, r2
```

—

4 Examples

Example 1

Input

```
int x;  
x = 5;
```

Output TAC

```
x = 5
```

Example 2

Input

```
int a;  
int b;  
a = 2;  
b = 3;
```

Output TAC

```
a = 2  
b = 3
```

Example 3

Input

```
x = 5 + 3;
```

Output TAC

```
t0 = 5 + 3  
x = t0
```

Example 4

Input

```
print(10);
```

Output TAC

```
param 10  
call print
```

Example 5

Input

```
int x;  
x = 5;  
print(x);
```

Output TAC

```
x = 5  
param x  
call print
```

Example 6

Input

```
int x;  
x = 10;  
if (x > 5) {  
    print(x);  
}
```

Output TAC

```
x = 10  
if x > 5 goto L1  
goto L2  
L1:  
param x  
call print  
L2:
```

Example 7

Input

```
int x;  
x = 3;  
if (x > 2) {  
    print(x);  
} else {  
    print(0);  
}
```

Output TAC

```
x = 3  
if x > 2 goto L1  
goto L2  
L1:  
param x  
call print  
goto L3  
L2:  
param 0
```

```
call print
L3:
```

Example 8

Input

```
int i;
i = 0;
while (i < 3) {
    print(i);
    i = i + 1;
}
```

Output TAC

```
i = 0
L1:
if i < 3 goto L2
goto L3
L2:
param i
call print
t0 = i + 1
i = t0
goto L1
L3:
```

Example 9

Input

```
x = (2 + 3) * 4;
```

Output TAC

```
t0 = 2 + 3
t1 = t0 * 4
x = t1
```

Example 10

Input

```
int a;
a = 2 + 3;
```

Optimized TAC

```
a = 5
```

5 Viva Questions and Answers (Tanjim Tajwar Arnab)

5.1 Abstract Syntax Tree (AST)

Q1: What is an Abstract Syntax Tree (AST)? AST is a tree representation of a program where each node represents a construct such as an expression or statement.

Q2: Why is AST preferred over a parse tree? AST removes unnecessary grammar details and keeps only meaningful structure, making it easier for semantic analysis and code generation.

Q3: How did you design your AST node structure? I designed nodes with fields like type, value, and pointers to left and right children to represent hierarchical relationships.

Q4: How do you represent arithmetic expressions in AST? Each operator becomes a node, and operands become its children. For example, $5+3$ becomes an ADD node with children 5 and 3.

Q5: How are control flow constructs represented in AST? Structures like IF, IF_ELSE, and WHILE are nodes where the condition and body are stored as child nodes.

5.2 Semantic Analysis

Q6: What is semantic analysis in a compiler? Semantic analysis checks if the program logically makes sense, such as type correctness and proper variable usage.

Q7: How do you detect undeclared variables? When a variable is used, we check the symbol table. If it is not found, we report an error.

Q8: How do you detect multiple declarations in the same scope? Before inserting a variable into the symbol table, we check if it already exists in the current scope.

Q9: How do you implement scope handling? We use a stack-based symbol table where each block creates a new scope and is removed after exiting.

Q10: How do you handle type checking in expressions? We compare operand types during AST traversal. If types do not match, an error is reported.

5.3 Symbol Table Interaction

Q11: How does semantic analysis interact with the symbol table? Semantic analysis uses the symbol table to verify variable declarations, types, and scope rules.

Q12: What happens when a variable is used before declaration? The compiler throws a semantic error indicating the variable is undeclared.

Q13: What data structures are used in semantic analysis? We use trees for AST and a stack-based structure (or array/list) for symbol table management.

5.4 Three Address Code (TAC)

Q14: What is Three Address Code (TAC)? TAC is an intermediate representation where each instruction has at most three operands.

Q15: Why do we use intermediate code? It simplifies translation to target machine code and allows optimization.

Q16: How do you generate temporary variables? We use a counter to create names like `t0`, `t1`, `t2` for intermediate results.

Q17: How is an if-else statement converted into TAC? We use conditional jumps and labels. The condition decides which block (if or else) to execute.

Q18: How is a while loop translated into TAC? We use labels for loop start and end, along with conditional jumps to repeat execution.

5.5 Advanced / Integration

Q19: How does AST traversal help in code generation? We traverse the AST (usually recursively) to generate code for each node in correct order.

Q20: What challenges did you face while integrating all compiler phases? Ensuring smooth data flow between phases (lexer $\rightarrow$ parser $\rightarrow$ AST $\rightarrow$ semantic $\rightarrow$ codegen) was challenging, especially maintaining consistency in structure and error handling.

6 Viva Questions and Answers (Sabrina Sultana)

6.1 Lexical Analysis

Q1: What is lexical analysis? Lexical analysis is the first phase of a compiler that converts source code into tokens.

Q2: What is the role of a lexical analyzer? It scans the input program, identifies tokens, and passes them to the parser.

Q3: What are tokens? Tokens are meaningful units such as keywords, identifiers, numbers, and operators.

Q4: What is a lexeme? A lexeme is the actual character sequence in the source code that matches a token pattern.

Q5: How does Flex work? Flex takes regular expressions as input and generates a C program that acts as a lexical analyzer.

Q6: How do you define patterns in Flex? Patterns are defined using regular expressions that match tokens like numbers or identifiers.

Q7: How do you differentiate keywords and identifiers? Keywords are matched first with specific patterns; anything not matching becomes an identifier.

Q8: How do you handle whitespace? Whitespace such as spaces and tabs are ignored using rules that do not return tokens.

Q9: How do you handle comments? Comments are recognized by patterns (like `//` or `/* */`) and ignored during scanning.

Q10: What happens when an invalid character is found? The lexer reports a lexical error with the line number and skips the character.

Q11: How do you track line numbers in the lexer? We increment a counter whenever a newline character is encountered.

Q12: What is the output of the lexical analyzer? A sequence of tokens is passed to the syntax analyzer.

Q13: Why is lexical analysis important before parsing? It simplifies parsing by converting raw input into structured tokens.

Q14: Can lexical analysis detect syntax errors? No, lexical analysis only detects invalid tokens, not grammar errors.

Q15: What challenges did you face during lexical implementation? Handling edge cases like comments, whitespace, and distinguishing identifiers from keywords was challenging.

7 Viva Questions and Answers (Hafiz Hasnat Sifat Jami)

7.1 Syntax Analysis

Q1: What is syntax analysis? Syntax analysis is the phase of the compiler that checks whether the sequence of tokens follows the grammar rules of the language.

Q2: What is the role of a parser? The parser takes tokens from the lexical analyzer and verifies the structure of the program while building the Abstract Syntax Tree (AST).

Q3: What is a grammar in compiler design? A grammar defines the structure of a language using rules that describe how valid statements are formed.

Q4: What type of parser does Bison generate? Bison generates an LALR (Look-Ahead Left-to-Right) parser.

Q5: What is LALR parsing? LALR parsing is a type of bottom-up parsing technique that reads input from left to right and produces a rightmost derivation in reverse.

Q6: How do you define grammar rules in Bison? Grammar rules are defined using production rules where non-terminals are expanded into terminals or other non-terminals.

Q7: What are terminals and non-terminals? Terminals are actual tokens from the lexer, while non-terminals represent grammatical constructs.

Q8: How do you handle ambiguity in grammar? We resolve ambiguity using operator precedence and associativity rules defined in Bison.

Q9: What is operator precedence and associativity? Operator precedence defines which operator is evaluated first, while associativity defines the order when operators have the same precedence.

Q10: How do you handle syntax errors? We use Bison's error handling mechanism to detect syntax errors and print meaningful messages.

Q11: How is AST built inside the parser? AST nodes are created during parsing by attaching child nodes according to grammar rules.

Q12: What happens if the input is grammatically incorrect? The parser reports a syntax error and stops or recovers depending on the error handling strategy.

Q13: What are shift-reduce conflicts? A shift-reduce conflict occurs when the parser cannot decide whether to shift the next token or reduce a rule.

Q14: How did you resolve parser conflicts? We resolved conflicts using precedence rules and carefully designing grammar to remove ambiguity.

Q15: What challenges did you face while writing grammar? Designing unambiguous grammar and handling operator precedence correctly were the main challenges.

8 Viva Questions and Answers (Muznabin Ahmed)

8.1 Symbol Table

Q1: What is a symbol table? A symbol table is a data structure used by the compiler to store information about variables, such as their name, type, and scope.

Q2: Why is it important in a compiler? It helps the compiler track declared variables and ensures correct usage during semantic analysis.

Q3: What information do you store in the symbol table? We store variable name, type, and scope level.

Q4: What data structure did you use for symbol table implementation? We used an array or list structure along with a stack-based approach for scope management.

Q5: How do you insert variables into the table? When a variable is declared, we add its information to the symbol table.

Q6: How do you perform lookup operation? We search the table from the current scope upward to find the variable.

Q7: How do you handle scope using symbol table? We push a new scope when entering a block and pop it when leaving the block.

Q8: What happens when a variable is redeclared? An error is reported if the same variable is declared again in the same scope.

Q9: What happens when a variable is not found? The compiler reports a semantic error indicating an undeclared variable.

Q10: How do you manage block-level scoping? Each block creates a new scope, and variables inside it are only accessible within that block.

Q11: What is the time complexity of lookup? In a simple implementation, lookup can be $O(n)$ where n is the number of symbols.

Q12: How does symbol table help semantic analysis? It provides necessary information to check declarations, types, and scope rules.

Q13: What issues can occur without a symbol table? Without it, the compiler cannot detect undeclared variables, type errors, or scope violations.

Q14: How do you handle nested scopes? Nested scopes are handled using a stack where each new scope is pushed and removed after exit.

Q15: What challenges did you face implementing the symbol table? Managing scope correctly and ensuring efficient lookup across multiple scopes were the main challenges.

9 Viva Questions and Answers (Monir Hossain)

9.1 Optimization and Target Code Generation

Q1: What is code optimization? Code optimization is the process of improving intermediate code to make it more efficient without changing its output.

Q2: Why is optimization needed in a compiler? It reduces execution time and memory usage, making the program faster and more efficient.

Q3: What is constant folding? Constant folding is an optimization technique where constant expressions are evaluated at compile time.

Q4: Can you give an example of constant folding? For example, $x = 2 + 3$; can be optimized to $x = 5$;

Q5: What is dead code elimination? Dead code elimination removes code that does not affect the final output, such as unused variables or assignments.

Q6: How do you detect dead code? We analyze whether a variable is used after assignment. If not, the assignment is considered dead.

Q7: What is redundant code? Redundant code refers to unnecessary instructions that do not contribute to the final result.

Q8: How do you remove redundant temporaries? We track temporary variables and remove those that are not used later in the program.

Q9: What is target code generation? It is the process of converting intermediate code into low-level machine-like or assembly-like instructions.

Q10: Why do we convert TAC into target code? Because TAC is machine-independent, and target code is required for execution on a machine.

Q11: What type of target code did you generate? We generated pseudo assembly code using simple instructions like LOAD, ADD, and JMP.

Q12: What are registers in assembly? Registers are small storage locations inside the CPU used to hold temporary values during computation.

Q13: How do you map TAC instructions to assembly? Each TAC instruction is converted into a sequence of assembly instructions using registers.

Q14: How do you handle control flow in target code? We use labels and jump instructions to represent conditions and loops.

Q15: What challenges did you face during optimization or target generation?
Ensuring correctness while removing unused code and mapping TAC instructions properly to assembly was challenging.

10 Team Work Description and Contributions

This project was developed collaboratively by five members. Each member was responsible for implementing a specific phase of the compiler. The contributions are described below:

10.1 Tanjim Tajwar Arnab (ID: 22701066) – 40% Contribution

Implemented Components:

- Abstract Syntax Tree (AST)
- Semantic Analysis
- Three Address Code (TAC) Generation
- Integration of all compiler phases

What He Built: Tanjim designed the AST structure to represent the program in a tree format. He then implemented semantic analysis to ensure correctness of variables, types, and scope. Finally, he built the code generation module that converts AST into Three Address Code.

What It Does:

- AST organizes program logic hierarchically
 - Semantic analysis ensures correctness (type, scope, declaration)
 - TAC converts the program into intermediate representation
 - Integration connects all phases into one working compiler
-

10.2 Mesabrina Sultana (ID: 22701067)

Implemented Component:

- Lexical Analyzer (lexer.l)

What She Built: She developed the lexical analyzer using Flex, which reads the input source code and breaks it into tokens.

What It Does:

- Identifies keywords, identifiers, numbers, and operators
 - Removes whitespace and comments
 - Detects invalid characters
 - Sends tokens to the parser
-

10.3 Hafiz Hasnat Sifat Jami (ID: 22701068)

Implemented Component:

- Syntax Analyzer (parser.y)

What He Built: He designed the grammar rules using Bison and implemented the parser that validates program structure.

What It Does:

- Checks if the program follows correct grammar
 - Builds the AST during parsing
 - Handles operator precedence
 - Detects syntax errors
-

10.4 Muznabin Ahmed (ID: 22701069)

Implemented Component:

- Symbol Table (symbol_table.c)

What He Built: He created the symbol table to store information about variables such as name, type, and scope.

What It Does:

- Stores variable declarations
 - Supports lookup and insertion operations
 - Manages scope using stack-based approach
 - Helps semantic analysis check variable correctness
-

10.5 Monir Hossain (ID: 21701009)

Implemented Components:

- Code Optimization
- Target Code Generation

What He Built: He implemented optimization techniques and developed the module to convert TAC into pseudo assembly code.

What It Does:

- Performs constant folding
 - Removes dead or redundant code
 - Converts TAC into low-level instructions
 - Handles labels and control flow for execution
-

10.6 Overall Contribution

Although each member had specific responsibilities, the project was developed collaboratively. All members participated in testing, debugging, and ensuring proper integration between compiler phases.

The final outcome is a complete MiniLang compiler that successfully translates source code into optimized target code.